Phase 2: Data Preparation Pipeline

Data Loading and Initial Cleaning

The real-world datasets can have system errors that result in the same transaction being recorded multiple times. This module guarantees that you have unique customers.

Logic Flow

1. Load the unprocessed CSV.
2. Find the rows with the same Customer_ID.
3. Keep the first one and eliminate the duplicates.

Usage Context

This is important for the churn prediction or KNN cases where duplicated entries would add an artificial "weight" to some data points which would lead to overfitting.

Technical Breakdown

- `import pandas as pd`: Imports the main library utilized for data structure and manipulation (DataFrames).
- `import numpy as np`: Gives support for big, multi-dimensional and mathematical arrays.
- `from sklearn.preprocessing import MinMaxScaler`: Imports the function used to compress numbers to a lower and more even range.
- `df = pd.read_csv('/adjusted_noisy_knn_300.csv')`: Imports the source file to the local.
- `df = df.drop_duplicates(subset=['Customer_ID'])`: Checks the ID column, finds duplicates, and deletes the rows that come after them.

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler

df = pd.read_csv('/adjusted_noisy_knn_300.csv')

df = df.drop_duplicates(subset=['Customer_ID'])
```

Noise Filtering & Median Imputation

Datasets can sometimes include data which is considered "garbage" (i.e., ages that are -5 or 200). Instead of deleting entire rows and discarding other important information, we mask such errors, and fill them with averages.

Logic Flow

1. Specify "impossible" age ranges.
2. Set those values to NaN.
3. To uphold statistical central tendency, fill NaN with Median of column.

Usage Context

In this way we avoid results that are skewed. Using Median is more protective than Mean since Median is more robust to outliers (like 1 billionaire in a dataset of middle-class earners).

Technical Breakdown

- `df.loc[(df['Age'] > 100) | (df['Age'] < 0), 'Age']`: Applies boolean indexing to check for ages below 0, above 100 and mark them as null.
- `df.loc[df['Annual_Income'] < 0, 'Annual_Income'] = np.nan`: Identifies values of income that are negative and marks them as null.
- `df['Age'] = df['Age'].fillna(df['Age'].median())`: Fills in null values with middle value of Age column.
- `df['Annual_Income'] = df['Annual_Income'].fillna(df['Annual_Income'].median())`: Fills null values in income with median value of income.
- `df['Total_Spend'] = df['Total_Spend'].fillna(df['Total_Spend'].median())`: Fills gaps in spending history with median.

```
df.loc[(df['Age'] > 100) | (df['Age'] < 0), 'Age'] = np.nan
df.loc[df['Annual_Income'] < 0, 'Annual_Income'] = np.nan

df['Age'] = df['Age'].fillna(df['Age'].median())
df['Annual_Income'] = df['Annual_Income'].fillna(df['Annual_Income'].median())
df['Total_Spend'] = df['Total_Spend'].fillna(df['Total_Spend'].median())
```

Categorical Standardization & Binning

Computers interpret "Male" and "male" as different labels. This part of the process describes the data and forces everything into a more structured format, also converting continuous variables into categorical variables.

Logic Flow

1. Whitespace is removed and case is standardized.
2. Expand abbreviations like "m" for male and "fem" to female.
3. `Items Purchased` is categorized into "Low, Medium, High" buckets.

Usage Context

This enhances the "signal" within the data. When the amount of purchases is grouped, it allows the model to identify patterns, rather than differentiate between thresholds like 14 and 15 purchased items.

Technical Breakdown

- `.str.strip().str.lower()` - removes whitespace (e.g., " Male"), and changes everything to lowercase to facilitate matching.
- `.replace({...})` - dictionary based mapping that categorizes a dataset to make it more manageable.
- `pd.cut(...)` - assigns values to "bins". Meaning that for Items_Purchased, it will return "Low" for 0-15, "Medium" for 15-35, and so on.

```
df['Gender'] = df['Gender'].str.strip().str.lower().replace({
    'm': 'Male', 'male': 'Male', 'f': 'Female', 'female': 'Female', 'fem': 'Female'
})
df['Device_Type'] = df['Device_Type'].str.strip().str.capitalize()

# Print columns to diagnose missing 'Item_Purchased' column
print("DataFrame Columns:", df.columns)

df['Purchase_Volume'] = pd.cut(df['Items_Purchased'],
                               bins=[0, 15, 35, 51],
                               labels=['Low', 'Medium', 'High'])
```

```
DataFrame Columns: Index(['Customer_ID', 'Age', 'Gender', 'Annual_Income', 'Purchase_Date',
       'Membership_Score', 'Total_Spend', 'Store_Region', 'Items_Purchased',
       'Last_Logon_Days', 'Promo_User', 'Device_Type', 'Support_Calls',
       'Newsletter_Sub', 'Purchase_Likelihood'],
      dtype='object')
```

External Data Merging (Enrichment)

It is common for us to need to incorporate external context for our primary dataset. In this case, we incorporate a tax rate associated with a customer's location.

Logic Flow

1. Create a reference table (Tax rates per region).
2. Do a `Left Join` to align/merge those rates with our existing customers.

Usage Context

Feature Engineering. With Tax_Rate, we are signaling to the model that a customer's spending habits may be affected by the tax rate in that region.

Technical Breakdown

- `pd.DataFrame({...})`: Creates a small manual table of regions and their corresponding tax percentages.
- `df = pd.merge(df, region_tax_data, on='Store_Region', how='left')`: In this case, Store_Region of our primary dataset is matched with Store_Region of the added table and Tax_Rate is "attached" to the customer row.

```
region_tax_data = pd.DataFrame({
    'Store_Region': ['North', 'South', 'East', 'West'],
    'Tax_Rate': [0.05, 0.07, 0.06, 0.08]
})
df = pd.merge(df, region_tax_data, on='Store_Region', how='left')
```

Feature Scaling & One-Hot Encoding

Machine Learning models, especially KNN, determine the "distance" between data points. If the income is 50,000 and the age is 20, the distance is mostly determined by income. Scaling solves this problem.

Logic Flow

1. Rescale all values to a range between 0 and 1.
2. Turn text categories (North, South) into binary columns (Is_North: 1, Is_South: 0).

Usage Context

Necessary for distance-based algorithms. Otherwise, the model would assume Annual_Income is 1,000x more significant than Support_Calls.

Technical Breakdown

- `scaler=MinMaxScaler()`: This sets up the following formula: $x_{scaled} = fracx - minmax - min.$
- `scaler.fit_transform(...)`: This computes the min and max for every column and performs the scaling.
- `pd.get_dummies(...)`: This is a method for models that don't "understand" a column. This method generates a column for each distinct category and uses 1 and 0 to signal their presence.

```
scaler = MinMaxScaler()
numeric_cols = ['Age', 'Annual_Income', 'Total_Spend', 'Items_Purchased', 'Last_Logon_Days', 'Support_Calls', 'Tax_R
df[numeric_cols] = scaler.fit_transform(df[numeric_cols])

df = pd.get_dummies(df, columns=['Gender', 'Device_Type', 'Store_Region', 'Purchase_Volume'])
```

Cleanup and Final Export

In the end, we eliminate columns that are unnecessary (such as IDs) or that contain information that the model cannot understand (such as raw dates).

Logic Flow

1. Detect non-predictive columns.
2. Remove them from the DataFrame.
3. Export the cleaned file.

Usage Context

Constructing the final "feature matrix" for model training.

Technical Breakdown

- `cols_to_drop = [...]`: Identifies columns that do not add value to the mathematical predictions (e.g., Customer_IDs are just static identifiers, not behavior descriptors).
- `df.drop(...)`: Permanently deletes these columns from the final dataset.
- `df_final.to_csv(...)`: Saves the final dataset, eliminating the need to execute the cleaning script every time training a model.

```
cols_to_drop = ['Membership_Score', 'Newsletter_Sub', 'Customer_ID', 'Purchase_Date']
df_final = df.drop(columns=cols_to_drop)

# Save the prepared dataset
df_final.to_csv('cleaned_knn_data.csv', index=False)
print("Data Preparation Complete. Final shape:", df_final.shape)
```

```
Data Preparation Complete. Final shape: (286, 21)
```

Baseline Model

Library Import and Data Loading

This part introduces the environment and the unprocessed data.

Logic Flow

1. Import the relevant modules for managing data, splitting models, and assessing performance.
2. Import the unprocessed dataset into the system before any modification.

Usage Context

This is the first step in every machine learning pipeline. We utilize the "dirty" dataset in order to understand the extent of the model's performance before it is cleaned.

Technical Breakdown

- `import pandas as pd`: Imports the data frame and CSV read library.
- `from sklearn.model_selection import train_test_split`: Imports the functionality to randomly split data into train and test sets.
- `from sklearn.neighbors import KNeighborsClassifier`: Imports the KNN algorithm which classifies points based on the "votes" of the closest neighbors.

- `from sklearn.metrics import accuracy_score, confusion_matrix`: Imports functions for model performance evaluation.
- `df_raw = pd.read_csv(...)`: This reads the unprocessed data file into a DataFrame called df_raw.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix

# Load the adjusted dataset
df_raw = pd.read_csv('/adjusted_noisy_knn_300.csv')
```

Naive Feature Selection and Missing Value Removal

This part introduces the environment and the unprocessed data.

Logic Flow

1. Categorical columns (strings) are filtered out, because mathematical models need numbers.
2. Rows containing at least one missing value (NaN) are deleted.

Usage Context

This is a "quick and dirty" method. It is very common when trying to get a model up and running in a matter of seconds; however, it means that a lot of useful information is discarded (like Gender or Region).

Technical Breakdown

- `df_naive = df_raw.select_dtypes(include=['number'])`: This scans the dataset and retains columns with integer or float types, meaning names, dates, and categories are dropped.
- `.dropna()`: This deletes rows where at least one cell is empty. This way the KNN algorithm will not crash because of some null values.

```
# --- Baseline Preparation ---
# 1. Keep only numeric columns (this automatically drops string 'noise')
# 2. Drop rows with any remaining Nan values
df_naive = df_raw.select_dtypes(include=['number']).dropna()
```

Feature-Target Separation and Data Splitting

Prior to training the model, we need to identify the features (the parts the model 'looks at') and the target (the component the model is trying to predict).

Logic Flow

1. Define X as the inputs and y as the answer.
2. Divide the data into two parts: one for training the model and the other for testing the model.

Usage Context

This split is crucial to avoid 'cheating' (also called overfitting). A model will seem to perform perfectly if evaluated on the same data used to train it, but it will perform poorly in the real world.

Technical Breakdown

- `X_naive = df_naive.drop(columns=['Purchase_Likelihood'])`: Here, we create the input matrix by dropping the target column we want to predict.
- `y_naive = df_naive['Purchase_Likelihood']`: Here, we set aside the target variable (Did they buy? Yes/No).
- `train_test_split(...)`: Here, 80% of the data is assigned to X_train and 20% is assigned to X_test. The `random_state=42` makes the output consistent every time this code is run for a given random split.

```
# Separate Features (X) and Target (y)
X_naive = df_naive.drop(columns=['Purchase_Likelihood'])
y_naive = df_naive['Purchase_Likelihood']

# Split the data
X_train, X_test, y_train, y_test = train_test_split(X_naive, y_naive, test_size=0.2, random_state=42)
```

Model Training and Evaluation

Here is where the KNN algorithm learns the patterns in the baseline and the performance is gauged from there.

Logic Flow

1. KNN model is initialized with a defined number of neighbors (k=15).
2. The model is fitted with the training data.
3. Predictions are made using the test set and they are contrasted with the actual "true" answers.

Usage Context

The results here serve as a "Floor." We can estimate that any cleaning or scaling we do later will improve this accuracy score.

Technical Breakdown

- `KNeighborsClassifier(n_neighbors=15)`: Configures the model to consider the 15 neighbor points in the data that is closest to a new unclassified point to define the class of that new point.
- `knn_naive.fit(X_train, y_train)`: The "Learning" step. The model captures the mapping of the relationship between the features and the target.
- `y_pred = knn_naive.predict(X_test)`: The "Testing" step. The model examines the features of the test set and makes a guess about the outcome.
- `accuracy_score(...)`: Computes (Correct Predictions / Total Predictions).
- `confusion_matrix(...)`: Provides a canvas that summarizes True Positives, True Negatives, False Positives, and False Negatives.

```
# Train k-NN (k=5)
knn_naive = KNeighborsClassifier(n_neighbors=5)
knn_naive.fit(X_train, y_train)

# Evaluate
y_pred = knn_naive.predict(X_test)
print(f"Baseline Accuracy: {accuracy_score(y_test, y_pred):.2%}")
print("Baseline Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
```

```
Baseline Accuracy: 42.86%
Baseline Confusion Matrix:
 [[3 1]
 [7 3]]
```

Optimized Model Execution

Environment Setup & Data Loading

This module equips the workspace with further advanced visualization capabilities and loads your data 'cleaned' version.

Logic Flow

1. Import the libraries necessary for math and modeling and the professional-grade viz libraries (Seaborn/Matplotlib).
2. Load the dataset that is already scaled, encoded, and deduped.

Usage Context

Loading the file cleaned_knn_data.csv and not the raw one guarantees that the model will not be confused by varying scales (e.g., Income vs. Age) and by non-numeric text.

Technical Breakdown

- `import matplotlib.pyplot as plt & import seaborn as sns`: These are the standard libraries for heatmaps and other charts used to visualize the performance of the model.
- `from sklearn.metrics import classification_report`: this is used to get more detailed metrics like Precision, Recall, and F1-Score.
- `df_prepared = pd.read_csv(...)`: This accesses the CSV we wrote at the end of the modular cleaning script.

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# 1. Load the prepared dataset
# Replace with 'cleaned_knn_data.csv' or your specific file name
# Changed path from '/cleaned_knn_data.csv'
df_prepared = pd.read_csv('cleaned_knn_data.csv')
```

Integrity Safety Check

Machine learning methods such as KNN will simply "crash" (throw a ValueError) if they see even one empty cell. This section is a kind of an insurance policy.

Logic Flow

1. Check the whole dataset for any hidden NaN (Null) values
2. If any are found (maybe as a result of a bug while exporting + importing), fill them in immediately with the median of that column.

Usage Context

In operational situations, data can become "damaged" during the exchange of files. This check makes sure that the training pipeline is solid and will not get interrupted during the middle.

Technical Breakdown

- `df_prepared.isnull().values.any()`: This returns True if there is at least one missing value in the whole DataFrame.
- `df_prepared.fillna(df_prepared.median(numeric_only=True))`: A safety net that fills any remaining gaps with a value that is statistically "safe" middle ground.

```
# --- SAFETY CHECK: HANDLE REMAINING Nans ---
# This fixes the "ValueError: Input X contains Nans"
if df_prepared.isnull().values.any():
    print(f"Detecting {df_prepared.isnull().sum().sum()} missing values...")
```

```
    # Option A: Fill with median (Recommended to keep data)
    df_prepared = df_prepared.fillna(df_prepared.median(numeric_only=True))
    # Option B: Or drop them if very few remain
    # df_prepared = df_prepared.dropna()
    print("Missing values handled.")
```

```
Detecting 159 missing values...
Missing values handled.
```

Data Partitioning (Features vs. Target)

To aid in tracking, we separate the clues from the answers, and then divide the dataset into training and testing subsets.

Logic Flow

1. Set Purchase_Likelihood as the objective.
2. Assign the remaining columns (the selected features) to the variable X.
3. Perform a split on the dataset using a ratio of 80/20.

Usage Context

We set random_state=42 to the same value as in the baseline. This is important for a "fair fight" since this value will show that the improvements we register are related to more refined data cleaning and not simply a more favorable random distribution of the rows.

Technical Breakdown

- `X = df_prepared.drop(columns= [target_col])`: Removes the answer from the training dataset.
- `train_test_split(..., test_size=0.2)`: Determines that 20% of the data shall be used as a final "exam" for the model.

```
# 2. Separate Features (X) and Target (y)
# Ensure the target name matches your dataset
target_col = 'Purchase_Likelihood'
X = df_prepared.drop(columns=[target_col])
y = df_prepared[target_col]

# 3. Split the Data (80% Training, 20% Testing)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

Model Training & Evaluation

We have run the KNN model on the cleaned data and produced a report detailing its performance.

Logic Flow

1. Instantiate KNN with $k = 5$.
2. Train the model on the cleaned and scaled data.
3. Evaluate the model's predictions against the test set ground truth.

Usage Context

This is the first place to appreciate the ROI (Return on Investment) of your efforts on cleaning data. Typically, the accuracy score here should be considerably higher than Baseline Accuracy.

Technical Breakdown

- `knn.fit(X_train, y_train)`: The KNN model computes distances in space with the cleaned feature set.

- `classification_report(y_test, y_pred)`: Rather than returning just one accuracy score, it answers the question of how well the model distinguishes buyers from non-buyers.
- `confusion_matrix(...)`: quotas the number of times the model predicted "Purchase" incorrectly (False Positive).

```
# 4. Train the k-NN Model (Optimized Run with k=5)
knn = KNeighborsClassifier(n_neighbors=13)
knn.fit(X_train, y_train)

# 5. Evaluate the Model
y_pred = knn.predict(X_test)

print("\n--- Optimized k-NN Model Resuls ---")
print(f"Accuracy Socre: {accuracy_score(y_test, y_pred):.2%}")
print("\nConfusion Matrix")
print(confusion_matrix(y_test, y_pred))
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

```
--- Optimized k-NN Model Resuls ---
Accuracy Socre: 67.24%

Confusion Matrix
[[21 10]
 [ 9 18]]

Classification Report:
              precision    recall  f1-score   support

           0       0.70      0.68      0.69        31
           1       0.64      0.67      0.65        27

    accuracy                           0.67        58
   macro avg       0.67      0.67      0.67        58
weighted avg       0.67      0.67      0.67        58
```